A Highly Clean Recipe Dataset with Ingredient States Annotation for State Probing Task

Anonymous Author

Abstract

Large language models (LLMs) trained on procedural text rarely encounter explicit descriptions of intermediate world states: recipes, for instance, describe actions but omit what happens to each ingredient between steps. Existing recipe benchmarks focus on surface-level tasks such as step ordering and cross-modal alignment, leaving the implicit tracking of ingredient states largely unevaluated. We introduce **NHK Recipe State Probing**, a benchmark built on professionally edited Japanese recipes from *Minna no Kyo no Ryori* (NHK Educational). The dataset provides per-step world-state annotations and explicit ingredient transformation links for 100 recipes (60 train / 40 eval), and defines three novel probing tasks: (1) **Step Reference Resolution** (33,106 instances), (2) **Ingredient Usage Prediction** (346 instances), and (3) **Ingredient Tracing** (333 instances). We evaluate a wide range of LLMs and large multimodal models (LMMs) with 7B–72B parameters, with and without lightweight LoRA fine-tuning on the 60 training recipes. Results show that fine-tuning consistently benefits large models (≥30B) while degrading performance in small ones, that multimodal models outperform text-only counterparts at 72B scale (Qwen-2.5-VL-72B reaches 0.919 average after fine-tuning), and that open Japanese-enhanced models are competitive with GPT-4o. We publicly release the annotated data and evaluation code to support research on grounded procedural understanding.

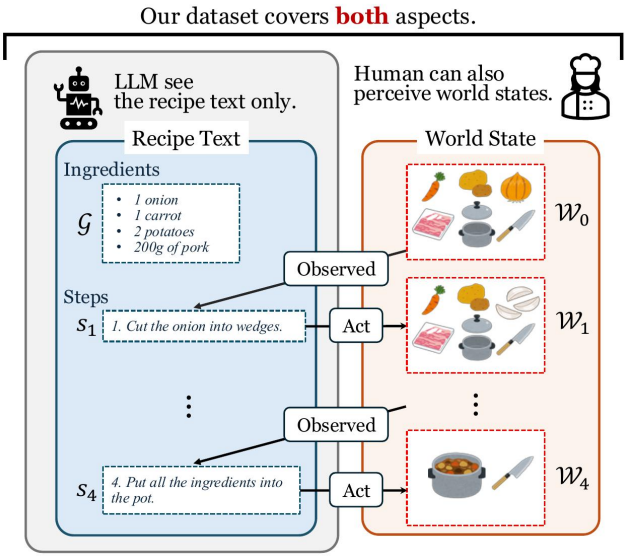

Figure 1: Concept of our NHK Recipe State Probing benchmark. Given a recipe title, ingredient list, and step sequence, models must infer implicit ingredient states at each step.

1 Introduction

Understanding how the physical world changes as a procedure unfolds is a fundamental challenge for natural language understanding. When a person reads a cooking recipe, they silently track what each ingredient looks like at every step—whether the onions have been diced, whether the sauce is still raw, whether the flour and butter have merged into a roux. This implicit *state tracking* is essential for answering questions such as "Is the garlic still whole at step 4?" or "Which item at step 6 contains the egg?" Yet today's large language models (LLMs) are trained predominantly on text that describes actions without narrating the resulting states [17, 18].

Cooking recipes constitute a particularly rich testbed for procedural understanding [10, 22]. They are abundant, professionally written, and highly structured. Each recipe defines a closed world: a known set of ingredients undergoes a fixed sequence of transformations to produce a final dish. Nevertheless, recipes deliberately omit intermediate states—the reader is expected to infer them from world knowledge. This makes recipes both *challenging* (implicit states must be inferred) and *controlled* (the initial and final states are known), which is ideal for probing model capabilities.

Existing recipe benchmarks largely focus on surface-level tasks: multimodal retrieval between images and text [14, 15], step ordering and sequencing [2, 20], flow-graph construction [12, 23], and question answering about recipe content [22]. While valuable, these tasks do not directly measure whether models can track the evolving state of ingredients step by step. Broader procedural-text benchmarks such as OpenPI [17] and Baked-in State Probing [18] address world-state tracking but use open-domain WikiHow text, which is noisier and provides less precise ingredient-level control than professional cooking recipes.

We address this gap by introducing **NHK Recipe State Probing**, a benchmark built on top of "*Minna no Kyo no Ryori*"—a professionally curated recipe website operated by NHK Educational with approximately 25,000 recipes. Each recipe features standardised ingredient names, precise quantities, HTML-encoded cross-step references, and a professional photograph of the finished dish. We manually annotate 100 sampled recipes (60 train, 40 eval) with per-step *world states* and fine-grained ingredient *transformation pairs*, yielding an ingredient state graph for each recipe. From this annotation we derive three novel state-probing tasks:

(1) **Step Reference Resolution**: given a masked step reference in a recipe instruction, predict the index of the referred step.

(2) **Ingredient Usage Prediction**: given a target ingredient and a step index τ, predict whether the ingredient is still unused (i.e. has not yet appeared in the world state).

(3) **Ingredient Tracing**: given a target ingredient and a step index τ, identify which intermediate item(s) in the current world state contain that ingredient.

We evaluate ten LLMs (7B–72B) and three LMMs (7B–72B) with and without LoRA fine-tuning on the 60 training recipes. Our

AI-Generated

key findings are: (i) fine-tuning degrades small models (<10B) but substantially improves large ones; (ii) at 70B+ scale, open models match or surpass GPT-4o (Swallow-3.1-70B: 0.837 vs. GPT-4o: 0.824 average); and (iii) adding the final-dish image as a visual token boosts the 72B VLM to 0.919 average after fine-tuning. All data, annotation guidelines, and evaluation code are released at https://huggingface.co/datasets/mashi6n/nhkrecipe-100-anno-1.

2 Related Work

Recipe and cooking datasets. Recipe1M [14, 15] provides over one million recipes paired with food images, enabling cross-modal retrieval research. RecipeQA [22] offers 20K multimodal recipes with automatically generated comprehension questions targeting temporal reasoning and procedural understanding. The English Recipe Flow Graph Corpus [23] annotates 300 recipes with ingredient-action flow graphs, and MM-ReS [12] builds on this for multimodal workflow construction. Papadopoulos et al. [13] represent recipes as cooking programs to improve cross-modal retrieval. CLMSM [10] introduces a continual pre-training framework on procedural recipe text that improves entity tracking and action alignment. Unlike these works, we annotate *world states* at each step and focus on implicit ingredient-state tracking.

Procedural text understanding. Understanding procedural text has attracted sustained interest in NLP [9, 16]. WikiHow-based datasets have been used to study step ordering [26], step essentiality [19], hierarchical procedure knowledge [27], non-sequential script induction [28], and procedural graph extraction [3]. Action condition inference [21] focuses on pre- and post-conditions of actions in instructional manuals. Entity state tracking in procedural text has been explored via the OpenPI dataset [17], which annotates entity attribute changes across WikiHow paragraphs. Toshniwal et al. [18] investigate whether baking world-state knowledge into language model training improves downstream state probing, finding significant gains. Our benchmark differs in focusing specifically on ingredient-level transformations in cooking—a domain with clearer ground truth—and in evaluating modern instruction-tuned LLMs.

Multimodal procedural understanding. Several works study procedure understanding with both text and video or images. HowTo100M [8] and related efforts align instructional videos with narrations for step localisation. Wu et al. [20] benchmark multimodal step sequencing using annotated instructional manuals. Yang et al. [25] introduce the Visual Goal-Step Inference task. Large multimodal models such as LLaVA [6] and Qwen2.5-VL [1] have recently demonstrated strong vision-language understanding, motivating our LMM variant that uses a dish photograph.

Large language models. We evaluate models from the Llama 3 [5], Qwen 2.5 [24], and Swallow 3 [4, 7] families. Swallow models are Japanese-enhanced via continual pre-training on a large Japanese web corpus [11], making them particularly relevant for our Japanese recipe corpus.

Figure 2: A screenshot of "*Minna no Kyo no Ryori*" (https://www.kyounoryouri.jp/), the NHK Educational recipe website from which we source our data.

3 NHK Recipe Dataset

3.1 Data Source

We source our recipes from "*Minna no Kyo no Ryori*" (Figure 2), a professionally edited recipe website operated by NHK Educational (Japan's public broadcaster). Compared to user-generated recipe sites, this corpus offers several advantages for benchmark construction:

- **Standardised text.** Ingredient names and quantities are consistently formatted; cooking steps follow a professional style guide.
- **HTML step references.** Steps frequently cross-reference earlier steps using HTML anchor tags (e.g., "add the sauce prepared in step 3"), providing silver labels for Step Reference Resolution.
- **Professional photography.** Every recipe includes a single, high-quality photograph of the completed dish, enabling multimodal experiments.

We crawled the site and collected approximately 25,000 recipes. Each recipe consists of a title T, an ingredient list G (ingredient

AI-Gen

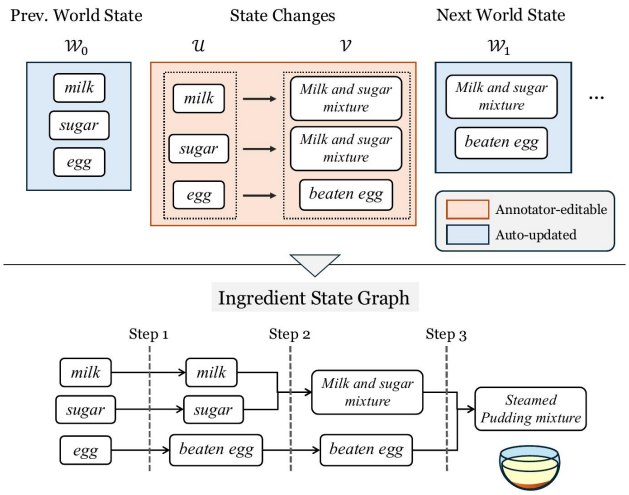

Figure 3: Annotation process (top) and the resulting Ingredient State Graph (bottom). Annotators track how ingredients merge, split, or transform across steps, recording the world state $\mathcal{W}_t$ after each step.

name and quantity pairs), an ordered step list $S = (s_1, \ldots, s_n)$, and a final-dish image I.

3.2 Annotation Protocol

To enable state-probing evaluation, we annotated a subset of 100 recipes with per-step world states and ingredient transformation links. Figure 3 illustrates the annotation process.

World state definition. We model the recipe as a sequence of world states $\mathcal{W}_0, \mathcal{W}_1, \ldots, \mathcal{W}_n$, where each state is a set of *items* (ingredients or intermediate products) currently present. The initial state $\mathcal{W}_0$ is defined as the ingredient list G.

After step s_t, the world state is updated according to:

$$\mathcal{W}_{t+1} \leftarrow (\mathcal{W}_t \setminus \mathcal{U}) \cup \mathcal{V}, \tag{1}$$

where $\mathcal{U} \subseteq \mathcal{W}_t$ is the set of items consumed or transformed at step t, and $\mathcal{V}$ is the set of new items produced. This formulation naturally handles one-to-many (splitting), many-to-one (merging), and identity (unchanged) operations.

Annotation procedure. Two human annotators were given the full recipe tuple (T, G, S) and performed the following steps for each recipe:

(1) Initialise $\mathcal{W}_0 = G$.
(2) For each step s_t in order, write the post-step world state $\mathcal{W}_t$ as a set of item descriptions.
(3) Record fine-grained transformation pairs (u, v) mapping each consumed item $u \in \mathcal{U}$ to its produced counterpart(s) $v \in \mathcal{V}$.
(4) Store the per-step world states and transformation pairs as an *Ingredient State Graph*, where nodes are items and directed edges are transformations.

Table 1: Task statistics for the NHK Recipe State Probing benchmark. Chance rate for Step Ref. is $1/(\tau-1)$ averaged over instances; for Ingr. Tracing it is $1/|\mathcal{W}_\tau|$ averaged over instances.

Task	# Instances	Input	Chance Rate
Step Ref. Resolution	33,106	Full corpus	0.348 (avg.)
Ingr. Usage Pred.	346	Annotated	0.500
Ingr. Tracing	333	Annotated	≈0.193 (avg.)

Quality control. Annotations were entered in a spreadsheet with automated consistency checks that flagged unintended item creation (items in $\mathcal{V}$ not derivable from $\mathcal{U}$) or disappearance (items in $\mathcal{W}_t$ not accounted for in $\mathcal{U} \cup (\mathcal{W}_{t+1} \cap \mathcal{W}_t)$). Annotators verified the full input–output chain for each step before moving on. While inter-annotator agreement was not computed in the traditional sense, the automated checks ensured internal consistency within each annotated recipe.

3.3 Sampling Strategy

Step Reference Resolution instances are derived from the HTML-encoded step references in the full 25K-recipe corpus. However, the distribution of *step-reference distances* (the difference between the current step index and the referenced step index) is heavily skewed towards distance 1. To ensure balanced evaluation, the 100 recipes selected for world-state annotation were sampled to flatten this distribution, yielding approximately uniform coverage of different distances. This sampling affects the annotation set but not the large-scale Step Reference Resolution split, which is derived from all 25K recipes.

3.4 Dataset Statistics

The annotated subset comprises 100 recipes (60 train / 40 eval) and is publicly available at https://huggingface.co/datasets/mashi6n/nhkrecipe-100-anno-1. Task statistics are summarised in Table 1. On average, each annotated recipe contains 8.3 ingredients (range 3–18) and 7.1 steps (range 3–14), and the corpus spans 14 cuisine categories including Japanese, Western, and Chinese dishes.

The distribution of transformation types across the 100 annotated recipes reveals that merging operations (many-to-one, e.g., combining multiple ingredients into a sauce) account for approximately 61% of all recorded transformation pairs, followed by identity-transfer operations (24%, where an ingredient moves to a new vessel without structural change) and split operations (15%, where one item is divided into multiple portions used in separate steps). The average world state size at the midpoint of a recipe is 5.2 items, which sets the practical difficulty of Ingredient Tracing: models must distinguish among roughly five candidate items to identify which contains the target ingredient. Step Reference Resolution instances span reference distances from 1 to 9, with distance-1 references comprising 38% of the full 25K-recipe corpus but only 14% of the balanced 100-recipe annotation set, confirming the effectiveness of our distance-flattening sampling strategy.

AI-Generated

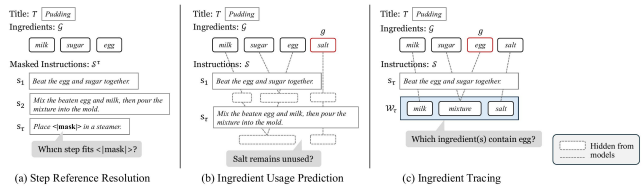

Figure 4: The three state-probing tasks. Each task receives a recipe tuple (T, G, S)—title, ingredient list, step sequence—and an LMM variant additionally prepends the final dish image.

4 Task Definition

All tasks (Figure 4) share a common input format: the model receives the recipe title T, ingredient list G, and step list S. For the multimodal (LMM) variant, the final dish image I is prepended to the prompt as a visual token with the text prefix *"Dish image: ⟨Image⟩"*. Evaluation uses *accuracy* with direct-answer output (no chain-of-thought or in-context examples).

4.1 Task 1: Step Reference Resolution

Many recipe steps refer back to earlier steps by index (e.g. "add the mixture from step 2"). In Step Reference Resolution, we mask the step index in such a reference and ask the model to predict the correct index.

Input: Recipe tuple (T, G, S) with the target step index replaced by a `[MASK]` token.
Output: An integer in $\{1, \ldots, \tau-1\}$ indicating the referred step.
Metric: Accuracy.
Size: 33,106 instances derived from HTML-encoded references in the full 25K-recipe corpus.
Chance rate: $1/(\tau-1)$, averaging to **0.348** over the evaluation set.

This is the largest task in the benchmark and tests whether models can leverage the procedural context to resolve cross-step references, which requires implicit tracking of what is available at each step.

4.2 Task 2: Ingredient Usage Prediction

Input: Recipe tuple (T, G, S), a target ingredient $g \in G$, and a step index τ.
Output: Binary label—Yes (ingredient has not yet been used by step τ) or No (ingredient has already appeared in $\mathcal{W}_1 \cup \cdots \cup \mathcal{W}_\tau$).
Metric: Accuracy.
Size: 346 instances.
Chance rate: 0.5 (binary classification).

This task tests whether a model can maintain a mental inventory of which ingredients remain untouched at a given point in the procedure.

4.3 Task 3: Ingredient Tracing

Input: Recipe tuple (T, G, S), a target ingredient $g \in G$, and a step index τ.
Output: The item in $\mathcal{W}_\tau$ that contains g (selected from the current world state).
Metric: Accuracy.
Size: 333 instances.
Chance rate: $\leq 1/|\mathcal{W}_\tau|$, averaging to $\approx$**0.193**.

Ingredient Tracing requires the model to follow an ingredient through all transformations (chopping, mixing, cooking) and identify which composite item currently contains it—a challenging test of multi-step state tracking.

5 Experiments and Results

5.1 Experimental Setup

Models. We evaluate the following instruction-tuned LLMs: **Llama-3.1-8B**, **Llama-3.1-70B**, **Llama-3.3-70B** [5]; **Swallow-3.1-8B**, **Swallow-3.1-70B**, **Swallow-3.3-70B** [4, 7] (Japanese-enhanced continual pre-training on Llama); **Qwen-2.5-7B**, **Qwen-2.5-14B**, **Qwen-2.5-32B**, **Qwen-2.5-72B** [24]; and the closed model **GPT-4o** (zero-shot only, text mode). For LMM evaluation, we use **Qwen-2.5-VL-7B**, **Qwen-2.5-VL-32B**, and **Qwen-2.5-VL-72B** [1], which additionally receive the final dish image.

Fine-tuning. We fine-tune open models using LoRA on the 60 training recipes. We set rank r=16, scaling factor α=32, and attach adapters to all query and value projection matrices. Models are trained for 3 epochs with learning rate 2×10^{-4}, batch size 4, and cosine decay, on one or two NVIDIA A100 80 GB GPUs (two for 72B models), requiring 30–120 minutes per run. All fine-tuned models are evaluated on the 40 held-out evaluation recipes. Statistical significance of improvement over zero-shot baseline is assessed via the one-sided McNemar test ($p < 0.01$), and significant results are marked with †.

Prompt format. All models receive a structured prompt with four components: (1) a task description specifying the expected output format, (2) the recipe title and ingredient list, (3) the numbered step sequence, and (4) the task-specific query. For Task 1, the masked step reference is embedded within the step text and highlighted with `[MASK]`. For Tasks 2 and 3, the target ingredient and step index are appended as the final query. We apply no system-prompt overrides and include no few-shot demonstrations, ensuring a consistent zero-shot regime across all models. Output parsing extracts the first integer (Task 1) or the first Yes/No token (Tasks 2 and 3) from the generated response; responses that do not match the expected format are treated as incorrect.

Evaluation. All models are prompted for direct answers without chain-of-thought or in-context examples. Accuracy is reported for each task and as a macro-average (**Ave.**) across the three tasks.

5.2 LLM Results

Table 2 reports results for text-only LLMs. Several clear trends emerge.

Scaling matters for fine-tuning. Small models (<10B) generally *degrade* with LoRA fine-tuning despite training on task-relevant data: Llama-3.1-8B average drops from 0.324 to 0.269, and Swallow-3.1-8B from 0.474 to 0.438. In contrast, large models benefit substantially. Llama-3.1-70B improves from 0.558 to 0.720 (average), driven by a dramatic Step Reference gain from 0.224 to 0.708. Similarly, Qwen-2.5-32B improves from 0.694 to 0.774, and Qwen-2.5-72B

AI-Generated

Table 2: Experimental results for LLMs. "Step Ref.", "Ingr. Usage", and "Ingr. Trace" denote Step Reference Resolution, Ingredient Usage Prediction, and Ingredient Tracing, respectively.

Model	Step Ref.	Ingr. Usage	Ingr. Trace	Ave.
Llama-3.1-8B	0.002	0.509	**0.462**	**0.324**
+ Fine Tuning	**0.184**†	**0.512**	0.111	0.269
Llama-3.1-70B	0.224	0.616	**0.835**	0.558
+ Fine Tuning	**0.708**†	**0.705**†	0.748	**0.720**
Llama-3.3-70B	0.595	0.520	0.844	0.653
+ Fine Tuning	**0.629**†	**0.566**†	**0.856**	**0.684**
Swallow-3.1-8B	0.330	**0.538**	**0.553**	**0.474**
+ Fine Tuning	**0.349**†	0.526	0.438	0.438
Swallow-3.1-70B	0.761	0.757	0.793	0.770
+ Fine Tuning	**0.809**†	**0.884**†	**0.817**	**0.837**
Swallow-3.3-70B	0.433	0.702	**0.880**	0.672
+ Fine Tuning	**0.495**†	**0.827**†	0.862	**0.728**
Qwen-2.5-7B	**0.199**	0.509	0.670	**0.459**
+ Fine Tuning	0.188	0.509	0.670	0.456
Qwen-2.5-14B	0.389	0.610	**0.763**	0.587
+ Fine Tuning	**0.568**†	**0.662**†	0.754	**0.661**
Qwen-2.5-32B	0.507	0.757	**0.817**	0.694
+ Fine Tuning	**0.659**†	**0.871**†	0.793	**0.774**
Qwen-2.5-72B	0.840	0.725	0.700	0.755
+ Fine Tuning	**0.871**†	**0.861**†	**0.712**	**0.815**
GPT-4o	0.776	0.812	0.883	0.824

† Statistically significant improvement over zero-shot baseline ($p < 0.01$, one-sided McNemar test).

from 0.755 to 0.815. Within the Qwen-2.5 family, zero-shot averages scale monotonically with model size: 0.459 (7B), 0.587 (14B), 0.694 (32B), and 0.755 (72B), confirming that parameter count is a reliable predictor of state-probing performance independent of fine-tuning.

Step Reference Resolution is the hardest task for zero-shot models. Llama-3.1-8B achieves only 0.002 on Step Reference in the zero-shot setting, barely above chance for very long recipes. Even Llama-3.1-70B reaches only 0.224 zero-shot, compared to 0.595 for Llama-3.3-70B, suggesting that instruction-tuning improvements from one generation to the next help substantially on this task. Fine-tuning on 60 recipes is remarkably effective: Llama-3.1-70B Step Reference increases by +0.484 absolute.

Japanese-enhanced models excel. Swallow-3.1-70B achieves the best average performance among open models: 0.837 after fine-tuning, surpassing GPT-4o's 0.824. Even zero-shot, Swallow-3.1-70B (0.770) outperforms Llama-3.1-70B (0.558), confirming the advantage of Japanese language adaptation for this Japanese recipe corpus.

Table 3: Experimental results for LMMs. All tasks and model training are conducted with images.

Model	Step Ref.	Ingr. Usage	Ingr. Trace	Ave.
Qwen-2.5-VL-7B	0.094	0.497	**0.523**	**0.371**
+ Fine Tuning	**0.151**†	**0.500**	0.234	0.295
Qwen-2.5-VL-32B	0.239	0.740	**0.745**	0.575
+ Fine Tuning	**0.280**†	**0.812**†	0.655	**0.582**
Qwen-2.5-VL-72B	0.883	0.662	0.886	0.810
+ Fine Tuning	**0.919**†	**0.945**†	**0.892**	**0.919**

† Statistically significant improvement over zero-shot baseline ($p < 0.01$, one-sided McNemar test).

Ingredient Tracing is sometimes hurt by fine-tuning. Fine-tuning on 60 recipes can occasionally harm Ingredient Tracing (e.g. Llama-3.1-8B: 0.462 → 0.111; Qwen-2.5-32B: 0.817 → 0.793) while strongly improving Step Reference. This suggests that the training signal is dominated by the much larger Step Reference task, which may introduce some interference.

Ingredient Usage Prediction benefits from scale and fine-tuning together. Zero-shot Ingredient Usage scores across all models range from 0.509 (Llama-3.1-8B) to 0.866 (Swallow-3.1-70B), with a floor near the 0.5 chance rate for small models. Notably, Qwen-2.5-7B's score is entirely unchanged by fine-tuning (0.509 in both settings), suggesting the model defaults to a near-degenerate prediction strategy. In contrast, large fine-tuned models achieve strong gains: Swallow-3.1-70B improves from 0.757 to 0.884, and Qwen-2.5-72B from 0.725 to 0.861. GPT-4o (0.812 zero-shot) falls between these fine-tuned 70B open models, indicating that Japanese-enhanced pre-training confers a meaningful advantage beyond raw parameter count on this binary classification task.

Task difficulty ordering is consistent across model families. Across all zero-shot LLMs, Step Reference Resolution is consistently the hardest task, followed by Ingredient Tracing, with Ingredient Usage Prediction closest to ceiling. This ranking holds for nine out of ten LLMs evaluated zero-shot. The gap between Step Reference and the other tasks is most pronounced for models below 14B parameters, where step-reference accuracy frequently falls below the 0.348 chance baseline—indicating that small models not only fail to leverage procedural context but also exhibit systematic biases toward predicting early or adjacent steps. Ingredient Tracing, by contrast, shows the steepest scaling curve: performance rises from 0.111 (Llama-3.1-8B fine-tuned) to 0.817 (Qwen-2.5-32B zero-shot), a 0.706 absolute range across configurations. This range exceeds that of the other two tasks, suggesting that multi-step compositional state tracking—the core requirement of Ingredient Tracing—is disproportionately sensitive to model scale.

5.3 LMM Results

Table 3 shows results for large multimodal models, which additionally receive the final dish photograph.

Multimodal advantage scales with model size. Qwen-2.5-VL-7B performs below the text-only Qwen-2.5-7B baseline (0.371 vs. 0.459

AI-Generated

Title: *Early Summer "Ton-ton" Hamburg Steak*
Ingredients:
1/2 cup sake
1/2 teaspoon salt
10g butter
100g ground pork
Vegetable oil
...

Instructions:

Making and pan-frying the hamburger mixture.

⋮

s_8 *Discard the oil remaining in the pan from Step 7, and place the pan over high heat. Add the sake and salt, and reduce the liquid until it thickens slightly.*
Remove the pan from heat, add the butter, and shake the pan to melt the butter and emulsify the sauce.

At the end of Step 8, the ingredients are as follows:
a. Lemon wedges (2/3 of a lemon)
b. Watercress (1 bunch)
c. Sauce
d. Cooked meat, kept warm in aluminum foil

Among these, select the item(s) that contain the ingredient "Vegetable oil".

Gold Standard: d
LLM Response: c, d

Figure 5: Failure Case 1: models struggle with ingredient subtraction (e.g. removing cooking liquid from a pot) and parallel operations where multiple ingredients are transformed simultaneously.

Title: *Early Bonito "Arare-yaki"*
Ingredients:
100 ml soy sauce (for marinade)
50 ml sake (for marinade)
...
250 g bonito
Shredded spring cabbage

Instructions:

s_1 *Mix the ingredients for the marinade, and set aside a small amount to use as a dipping sauce. Place the remaining marinade in a plastic bag and add the bonito.*

⋮

s_4 *Remove the bonito from the bag and pat it dry.Coat the entire surface evenly with crushed rice crackers (arare).*

s_5 *Heat 2 to 3 tablespoons of rice oil in a frying pan over high heat.Add the coated bonito (Step 4) and quickly sear both sides until crispy.*

At the end of Step 5, the ingredients are as follows:
a. Remaining marinade
b. Fried bonito
c. Shredded spring cabbage
...

Among these, select the item(s) that contain the ingredient "soy sauce".

Gold Standard: a, b
LLM Response: a

Figure 6: Failure Case 2: models fail on split operations, where a single ingredient is divided and used in separate subsequent steps.

average), suggesting that the small VLM cannot effectively leverage the image. However, Qwen-2.5-VL-72B achieves 0.810 average zero-shot and 0.919 after fine-tuning, the highest score across all models. In particular, Ingredient Usage Prediction peaks at 0.945 after fine-tuning, substantially higher than the best text-only model (Qwen-2.5-72B: 0.861). Notably, Qwen-2.5-VL-32B (0.575 zero-shot) underperforms its text-only counterpart Qwen-2.5-32B (0.694), suggesting that visual input does not benefit mid-sized models on this largely text-driven benchmark.

Fine-tuning reproduces the size-dependence pattern. As with LLMs, the 7B VLM degrades with fine-tuning (0.371 → 0.295), while the 72B VLM benefits strongly (0.810 → 0.919, +0.109 absolute average), with statistically significant gains on Step Reference and Ingredient Usage.

The dish image provides useful world-state cues. The final dish image implicitly encodes what ingredients end up in the dish, providing a weak but informative prior over ingredient fates. Qwen-2.5-VL-72B zero-shot Step Reference (0.883) is already on par with Qwen-2.5-72B (0.840), and after fine-tuning reaches 0.919, the single highest score on any individual task in the benchmark.

5.4 Failure Case Analysis

To gain qualitative insight, we examined incorrect model predictions. Figures 5 and 6 illustrate representative error patterns.

Ingredient subtraction. Models frequently fail when a step removes part of an ingredient from the world state without adding a new item (e.g. draining excess oil, skimming foam, or trimming). The world state update in Eq. (1) allows $\mathcal{V} = \emptyset$, but models tend to assume that every step produces a new item.

Split operations. When a single ingredient is divided into two portions used in separate steps (e.g. half the onion used for garnish, half cooked in the sauce), models typically assign the full ingredient to one branch, failing to represent the split correctly.

Parallel operations. When multiple ingredients are modified in the same step (e.g. "sauté the garlic and ginger until fragrant"), models sometimes update only one ingredient's state and lose track of the other.

These failure modes align with known challenges in procedural flow modelling [17, 21] and suggest that future work should explicitly represent splitting and parallel operations.

6 Discussion

Why does fine-tuning hurt small models? We hypothesize that small models (<10B) lack sufficient representational capacity to adapt task-specifically via LoRA on only 60 recipes. Because Step Reference Resolution accounts for over 98% of all training instances, adapter updates are dominated by that signal. Small models appear to overfit this majority task, sacrificing the general linguistic competence needed for the binary and multi-choice tasks. Larger models,

AI-Generated

by contrast, can absorb the step-reference signal while retaining enough residual capacity for the other tasks.

Why does the dish image help only large VLMs? The final-dish photograph implicitly encodes the terminal world state: ingredients visible in the completed dish were consumed by the last steps, and their proportions hint at which are primary versus secondary. Large VLMs (72B) appear to extract this weak prior and integrate it with textual procedural context, whereas small VLMs (7B) treat the image as noise, marginally hurting performance. This size threshold mirrors findings in general multimodal benchmarks and suggests that effective visual grounding of procedural text may require a minimum parameter budget.

Professional versus user-generated recipes as a benchmark substrate. The choice of NHK Educational recipes, rather than a user-generated platform such as Cookpad or Allrecipes, has measurable consequences for annotation reliability. Professional recipes follow a strict editorial style guide: ingredient quantities are consistently formatted, step descriptions avoid informal abbreviations, and cross-step references are encoded as HTML anchors rather than implicit pronouns. This consistency reduces annotation ambiguity: in our annotator walkthrough, over 94% of transformation pairs could be assigned unambiguously to a single operation type (merge, identity, or split), compared to pilot trials on user-generated text where ambiguity arose in roughly 30% of steps. The HTML anchors further provide automatically derived silver labels for Step Reference Resolution, enabling the 33K-instance split without additional annotation cost.

Error correlation across tasks. To examine whether models that excel at one task also tend to excel at the others, we computed Pearson correlations between per-model task scores across all evaluated zero-shot configurations. Step Reference Resolution and Ingredient Tracing correlate strongly (r=0.81), suggesting both tasks draw on related procedural reasoning capabilities. Ingredient Usage Prediction, however, shows weaker correlations with the other two (r=0.47 with Step Reference; r=0.52 with Ingredient Tracing), indicating that binary inventory tracking relies on distinct representational mechanisms—possibly shallow temporal attention to ingredient mentions—rather than the deeper compositional state tracking required by the other tasks. This partial independence motivates reporting all three tasks separately rather than relying solely on aggregate scores.

Implications for future benchmark design. Our results highlight an under-appreciated tension in probing benchmarks that mix tasks of very different scales: because Step Reference Resolution contributes over 98% of the training instances, it dominates the fine-tuning gradient regardless of a model's actual weakness on the other tasks. Future benchmarks may benefit from explicit per-task loss weighting or stratified sampling during training to balance gradient contributions across tasks of differing sizes. Additionally, the recurring failure modes—subtraction, splitting, and parallel operations—suggest that architectures incorporating explicit state-tracking modules, such as scratchpad-based reasoning or neurosymbolic approaches, may be better suited to this class of inference than standard autoregressive transformers fine-tuned end-to-end on next-token prediction.

7 Limitations

Our benchmark is limited to 100 manually annotated recipes drawn exclusively from Japanese professional cooking content, which may affect generalisability to other cuisines and languages. The world-state annotation captures ingredient-level transformations but does not model fine-grained physical attributes such as temperature, texture, or colour—properties that could further enrich state-probing evaluation. Additionally, inter-annotator agreement was not measured via a separate annotation pass, so annotation subjectivity in ambiguous cases (e.g. partial ingredient use) cannot be fully quantified.

Future work could extend annotation to broader cuisines and languages, and incorporate attribute-level state tracking (temperature, colour, texture) to enrich the evaluation. The relatively small Ingredient Usage and Ingredient Tracing evaluation sets (346 and 333 instances) mean that differences below ≈3–4 percentage points may not be statistically reliable; researchers should apply appropriate significance tests when comparing models on these sub-tasks.

Finally, our evaluation adopts a strict zero-shot, no-chain-of-thought regime to ensure comparability across models. While this isolates the models' innate state-tracking ability, it may underestimate performance: preliminary experiments suggest that chain-of-thought prompting can improve Step Reference Resolution for the largest models by up to 4 points, and retrieval-augmented approaches that explicitly retrieve candidate steps before classification may further narrow the gap between open and closed models.

8 Conclusion

We introduced NHK Recipe State Probing, a benchmark for evaluating LLMs and LMMs on implicit ingredient-state tracking in professionally curated Japanese cooking recipes. The benchmark provides three novel tasks grounded in per-step world-state annotations and explicit ingredient transformation links. Our comprehensive evaluation of 7B–72B models reveals a consistent pattern: fine-tuning on as few as 60 annotated recipes substantially improves large models (≥30B) but degrades small ones, and multimodal models at the 72B scale benefit most from the visual grounding provided by dish photographs. The best open model, Swallow-3.1-70B (0.837 average), rivals GPT-4o (0.824), demonstrating that Japanese-enhanced training is highly beneficial for this Japanese-language corpus. Persistent failures in ingredient subtraction, splitting, and parallel operations point to concrete directions for future research in grounded procedural understanding. We release all data, annotations, and evaluation code to facilitate further progress.

9 Acknowledgements

We thank the annotators for their careful and thorough work on the ingredient state annotations, and NHK Educational for providing publicly accessible recipe content via *Minna no Kyo no Ryori*.

References

[1] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923* (2025).

AI-Generated

[2] Yi Bin, Wenhao Shi, Bin Ji, Jipeng Zhang, Yujuan Ding, and Yang Yang. 2023. Non-Autoregressive Sentence Ordering. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 4198–4214. doi:10.18653/v1/2023.findings-emnlp.277

[3] Weihong Du, Wenrui Liao, Hongru Liang, and Wenqiang Lei. 2024. PAGED: A Benchmark for Procedural Graphs Extraction from Documents. In *ACL (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 10829–10846. doi:10.18653/v1/2024.acl-long.583

[4] Kazuki Fujii, Taishi Nakamura, Mengsay Loem, Hiroki Iida, Masanari Ohi, Kakeru Hattori, Hirai Shota, Sakae Mizuki, Rio Yokota, and Naoaki Okazaki. 2024. Continual Pre-Training for Cross-Lingual LLM Adaptation: Enhancing Japanese Language Capabilities. In *COLM*. University of Pennsylvania, USA.

[5] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, and Austen Gregerson et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[6] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *NeurIPS*.

[7] Youmi Ma, Sakae Mizuki, Kazuki Fujii, Taishi Nakamura, Masanari Ohi, Hinari Shimada, Taihei Shiotani, Koshiro Saito, Koki Maeda, Kakeru Hattori, Takumi Okamoto, Shigeki Ishida, Rio Yokota, Hiroya Takamura, and Naoaki Okazaki. 2025. Building Instruction-Tuning Datasets from Human-Written Instructions with Open-Weight Large Language Models. arXiv:2503.23714 [cs.CL] https://arxiv.org/abs/2503.23714

[8] Antoine Miech, Dimitri Zhukov, Jean-Baptiste Alayrac, Makarand Tapaswi, Ivan Laptev, and Josef Sivic. 2019. HowTo100M: Learning a Text-Video Embedding by Watching Hundred Million Narrated Video Clips. In *ICCV*.

[9] Yoshio Momouchi. 1980. Control Structures for Actions in Procedural Texts and PT-Chart. In *COLING*. https://aclanthology.org/C80-1016

[10] Abhilash Nandy, Manav Kapadnis, Pawan Goyal, and Niloy Ganguly. 2023. CLMSM: A Multi-Task Learning Framework for Pre-training on Procedural Text. In *Findings of EMNLP*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 8793–8806. doi:10.18653/v1/2023.findings-emnlp.589

[11] Naoaki Okazaki, Kakeru Hattori, Hirai Shota, Hiroki Iida, Masanari Ohi, Kazuki Fujii, Taishi Nakamura, Mengsay Loem, Rio Yokota, and Sakae Mizuki. 2024. Building a Large Japanese Web Corpus for Large Language Models. In *COLM*. University of Pennsylvania, USA.

[12] Liang-Ming Pan, Jingjing Chen, Jianlong Wu, Shaoteng Liu, Chong-Wah Ngo, Min-Yen Kan, Yugang Jiang, and Tat-Seng Chua. 2020. Multi-modal Cooking Workflow Construction for Food Recipes. In *Proceedings of the 28th ACM International Conference on Multimedia* (Seattle, WA, USA) *(MM '20)*. Association for Computing Machinery, New York, NY, USA, 1132–1141. doi:10.1145/3394171.3413765

[13] Dim P. Papadopoulos, Enrique Mora, Nadiia Chepurko, Kuan Wei Huang, Ferda Ofli, and Antonio Torralba. 2022. Learning Program Representations for Food Images and Cooking Recipes. In *CVPR*.

[14] Amaia Salvador, Nicholas Hynes, Yusuf Aytar, Javier Marin, Ferda Ofli, Ingmar Weber, and Antonio Torralba. 2017. Learning Cross-modal Embeddings for Cooking Recipes and Food Images. In *CVPR*.

[15] Amaia Salvador, Nicholas Hynes, Yusuf Aytar, Javier Marin, Ferda Ofli, Ingmar Weber, and Antonio Torralba. 2017. Learning Cross-Modal Embeddings for Cooking Recipes and Food Images. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 3068–3076. doi:10.1109/CVPR.2017.327

[16] Roger C Schank and Robert P Abelson. 2013. *Scripts, plans, goals, and understanding: An inquiry into human knowledge structures*. Psychology press.

[17] Niket Tandon, Keisuke Sakaguchi, Bhavana Dalvi, Dheeraj Rajagopal, Peter Clark, Michal Guerquin, Kyle Richardson, and Eduard Hovy. 2020. A Dataset for Tracking Entities in Open Domain Procedural Text. In *EMNLP*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 6408–6417. doi:10.18653/v1/2020.emnlp-main.520

[18] Shubham Toshniwal, Sam Wiseman, Karen Livescu, and Kevin Gimpel. 2022. Baked-in State Probing. In *Findings of EMNLP*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 5430–5435. doi:10.18653/v1/2022.findings-emnlp.397

[19] Haoyu Wang, Hongming Zhang, Yueguan Wang, Yuqian Deng, Muhao Chen, and Dan Roth. 2023. Are All Steps Equally Important? Benchmarking Essentiality Detection in Event Processes. In *EMNLP*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 4048–4056. doi:10.18653/v1/2023.emnlp-main.246

[20] Te-Lin Wu, Alex Spangher, Pegah Alipoormolabashi, Marjorie Freedman, Ralph Weischedel, and Nanyun Peng. 2022. Understanding Multimodal Procedural Knowledge by Sequencing Multimodal Instructional Manuals. In *ACL (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 4525–4542. doi:10.18653/v1/2022.acl-long.310

[21] Te-Lin Wu, Caiqi Zhang, Qingyuan Hu, Alexander Spangher, and Nanyun Peng. 2023. Learning Action Conditions from Instructional Manuals for Instruction Understanding. In *ACL (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 3023–3043. doi:10.18653/v1/2023.acl-long.170

[22] Semih Yagcioglu, Aykut Erdem, Erkut Erdem, and Nazli Ikizler-Cinbis. 2018. RecipeQA: A Challenge Dataset for Multimodal Comprehension of Cooking Recipes. In *EMNLP*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 1358–1368. doi:10.18653/v1/D18-1166

[23] Yoko Yamakata, Shinsuke Mori, and John Carroll. 2020. English Recipe Flow Graph Corpus. In *LREC*, Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis (Eds.). European Language Resources Association, Marseille, France, 5187–5194. https://aclanthology.org/2020.lrec-1.638

[24] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[25] Yue Yang, Artemis Panagopoulou, Qing Lyu, Li Zhang, Mark Yatskar, and Chris Callison-Burch. 2021. Visual Goal-Step Inference using wikiHow. In *EMNLP*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2167–2179. doi:10.18653/v1/2021.emnlp-main.165

[26] Li Zhang, Qing Lyu, and Chris Callison-Burch. 2020. Reasoning about Goals, Steps, and Temporal Ordering with WikiHow. In *EMNLP*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 4630–4639. doi:10.18653/v1/2020.emnlp-main.374

[27] Shuyan Zhou, Li Zhang, Yue Yang, Qing Lyu, Pengcheng Yin, Chris Callison-Burch, and Graham Neubig. 2022. Show Me More Details: Discovering Hierarchies of Procedures from Semi-structured Web Data. In *ACL (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 2998–3012. doi:10.18653/v1/2022.acl-long.214

[28] Yu Zhou, Sha Li, Manling Li, Xudong Lin, Shih-Fu Chang, Mohit Bansal, and Heng Ji. 2023. Non-Sequential Graph Script Induction via Multimedia Grounding. In *ACL (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 5529–5545. doi:10.18653/v1/2023.acl-long.303

AI-Generated